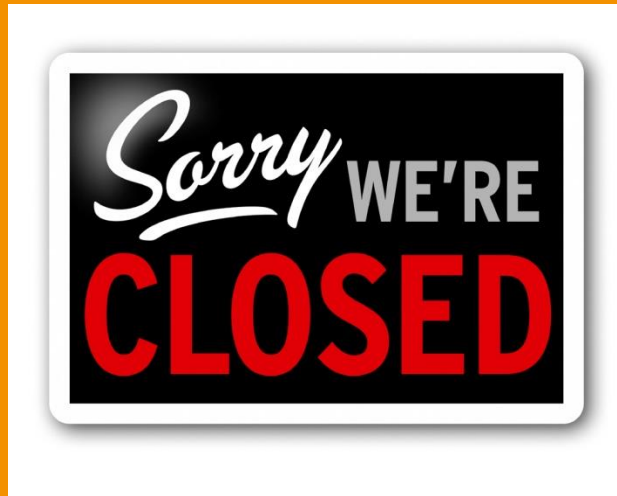




IT-Security

Chapter 4: Authentication and Authorization



1. Authentication
2. Single Sign On
3. Authorization



What do we want to learn?



- ▶ What is the difference between authentication and authorization?
authentication: identify person
authorization: identify what that person is allowed to do
- ▶ What variants are there for authentication?
- ▶ What variants are there for authorization?
- ▶ What all needs to be taken into account during practical implementation?



Chapter 4.1: Authentication

- Passwords
- Biometry
- Challenge Response
- Two Factor Authentication



Identification, Authentication, Authentication and Authorization

This distinction is important!



Identification
provide identity



Authentication
assert identity



Authentication
validate the assertion



Authorization
execute an access
policy



Definitions for Authentication and Authorization

▶ **Authentication:**

Authentication is the process of **verifying the identity** of a user, process, or device, often as a prerequisite to allowing access to resources in an information system. (NIST)

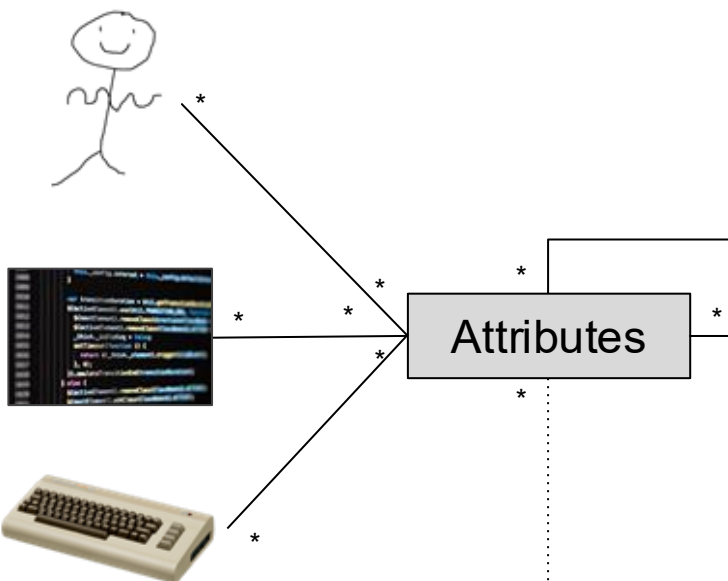
▶ **Authorization:**

Authorization is the process of checking whether a person, IT component or application is **authorized to perform a specific action or access a resource.**



Distinguish between identity and policy

who
Identity enables
authentication



what a person can do
Policy enables
authorization



Variants for authentication



- ▶ Authentication is possible through combinations are possible
 - ▶ knowledge
 - knowledge + possession: smartphone + PIN/ PWD
 - ▶ possession of e.g. a token
 - chip card (e.g. bank account)
 - ▶ personal characteristic
 - knowledge + personal char: spoken password (voice recognition)
 - possession + personal characteristic: chipcard + fingerprint, passport
- ▶ Password procedure
- ▶ Biometry
- ▶ Challenge-Response method
- ▶ Certificates / signatures
 - ▶ based on a PKI
 - > sign a random value/challenge -> verify with public key



Password authentication



- ▶ Widely used, easy to implement, mobile
- ▶ Problems
 - ▶ password cracking
 - ▶ if someone knows the password, he has unrestricted access
 - ▶ forgetting passwords → secure recovery procedure is required
 - ▶ unencrypted transmission over networks
- ▶ Measures to increase security
 - ▶ minimum length, special characters, regular changes???
 - ▶ blocking/delaying the identifier after a small number of failed attempts
 - ▶ display of last login, display unsuccessful login attempts
 - ▶ storage as hash value
 - ▶ use of a salt (randomly selected bit sequence) in the calculation of the hash value
- ▶ Additional measures
 - ▶ One-time passwords, TAN (transaction numbers), e.g. online banking
 - ▶ NONCE, captcha, security question, time stamp

▶ Password Manager

- ▶ A password manager is a tool that securely **stores, manages** and **automatically fills in** your password
- ▶ Advantages
 - ▶ You only need to remember one password (for the Password Manager)
 - ▶ Generates random, complex unique passwords
 - ▶ Automatically fills in login forms
 - ▶ Synchronization across devices
 - ▶ Alerts for weak or leaked passwords
 - ▶ Can be combined with 2FA (Two Factor Authentication)
- ▶ Examples
 - ▶ KeePassXC <https://keepassxc.org/>
 - ▶ Bitwarden <https://bitwarden.com>
 - ▶ 1Password <https://1password.com>
 - ▶ Keeper <https://www.keepersecurity.com>





Biometric techniques

- ▶ Fingerprint, face recognition, hand recognition, iris, retina, typing behavior, voice, signature recognition.
- ▶ Enable unique identification of individuals
- ▶ Problems
 - ▶ can be forged, intercepted, violently misused
 - ▶ import of biometric data bypassing the biometric sensor
 - ▶ some biometric characteristics cannot be exchanged
 - ▶ unnoticed collection and area-wide monitoring possible
 - ▶ Identity in reference database can be manipulated
 - ▶ pretend 100% security





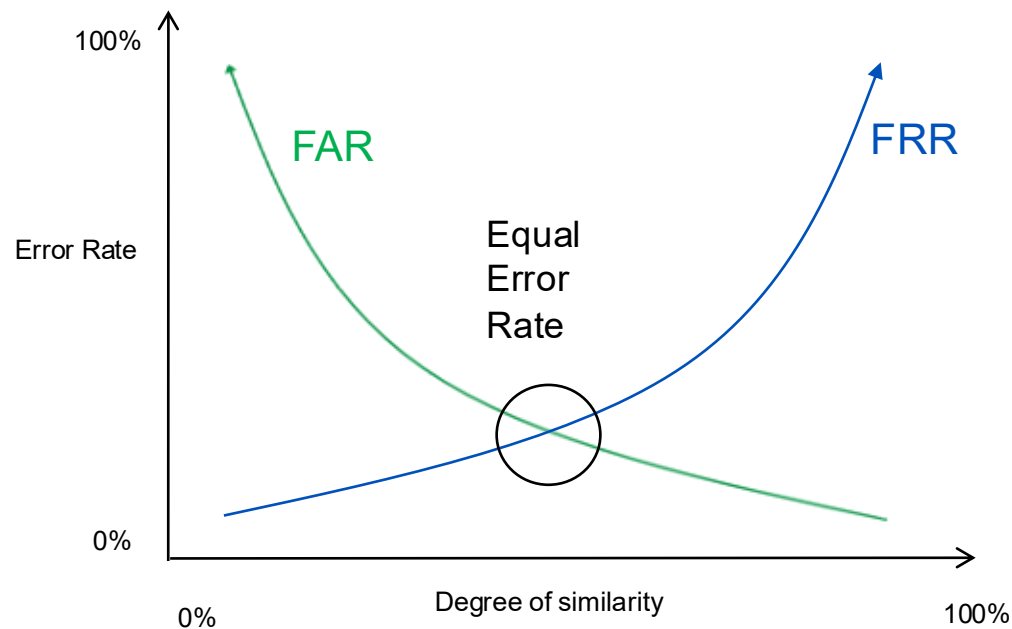
There are two problems with the verification of biometric authentication

▶ False Acceptance Rate FAR

- ▶ Falsely accepting an unauthorized person (false positive)

▶ False Rejection Rate FRR

- ▶ Falsely rejecting an authorized person (false negative)





Challenge Response Method

question
captcha (identify if person is human)

possession

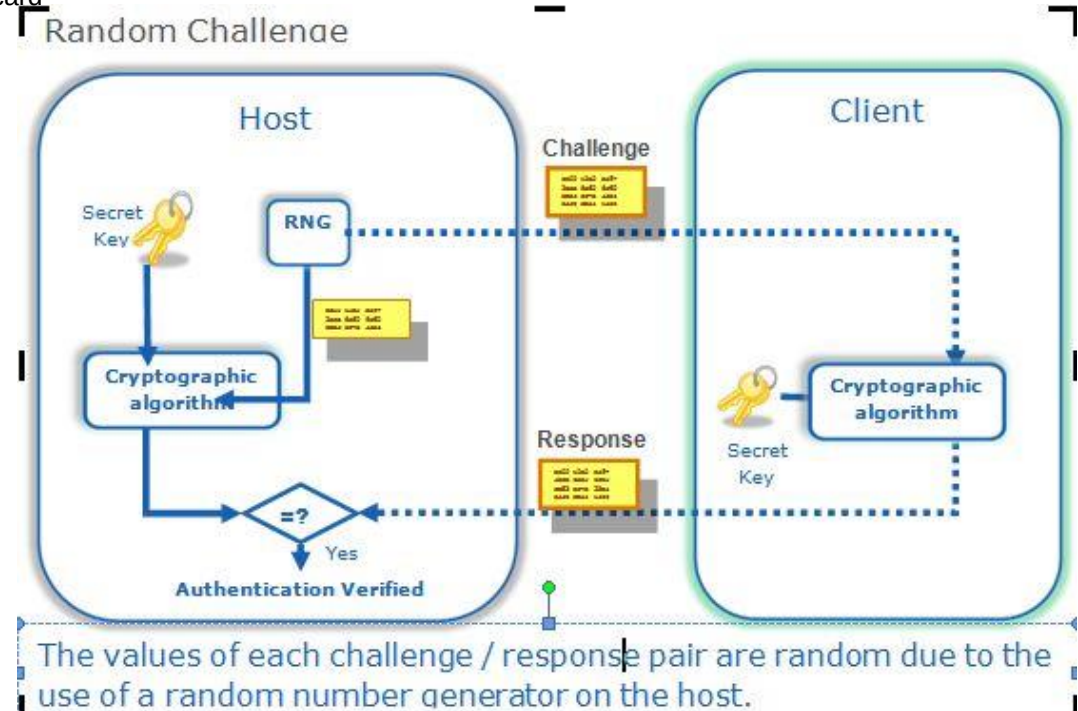
- Secure authentication method of a participant based on knowledge.
- One participant sets a task that the other must solve in order to prove that he knows a certain piece of information without transmitting this information himself.

pub-priv key pair also possible instead of just priv -> smart card

- Example: Challenge response based on a shared private key

- sender sends a random message to a receiver (challenge).
- the receiver encrypts it with a secret key and sends it back (Response)
- sender checks the result with its own encrypted version
- use case: Authentication of smart cards against workplace computers

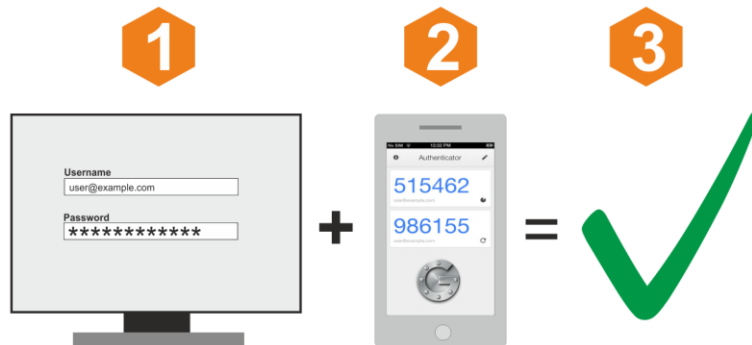
VLAN



Source: <https://atmelcorporation.wordpress.com/2013/04/01/random-challenge-response-authentication-in-plain-english/>

▶ Which authentication method do I use?

- ▶ Today, **Two-Factor Authentication (2FA)** is increasingly required
 - ▶ Main reason: Password method is too weak for today's computer power and password cracking techniques
 - ▶ Second factor is often a **One Time Token (OTT)** which is transmitted over another channel (**Two Channel Authentication**)
 - ▶ Security features of the second factor: unique, valid for a limited period of time
 - ▶ Examples: RSA Secure ID, Yubikey, TAN, SMS, special mobile apps (e.g. Google Authenticator)



Source: https://docs.opnsense.org/manual/two_factor.html



Source: <https://www.yubico.com/de/product/yubikey-5-nfc/>



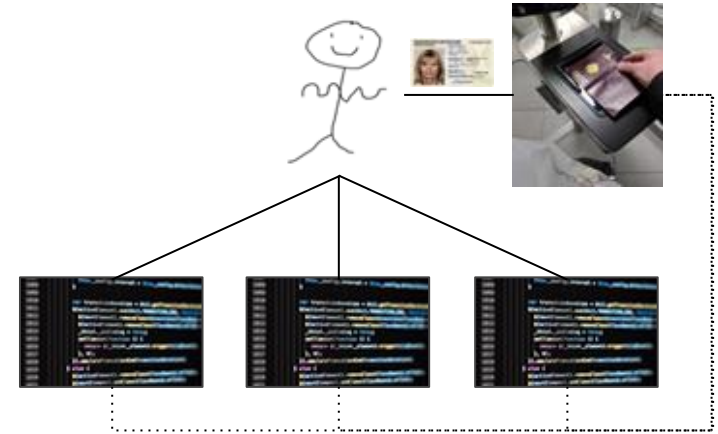
Chapter 4.2: Single Sign On

- SSO
- Kerberos
- SAML
- OAuth and OpenID



Single Sign On (SSO)

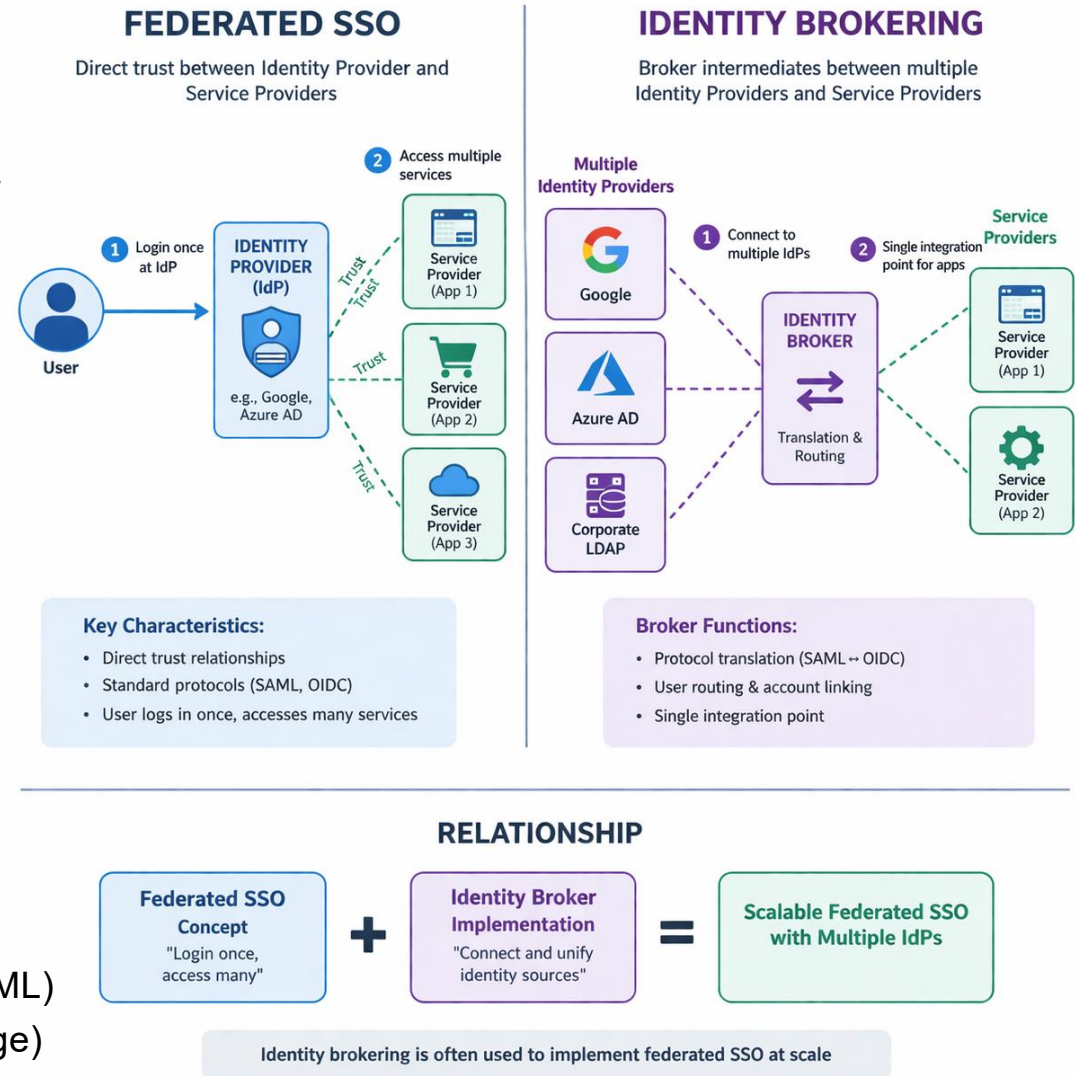
- ▶ After a one-time authentication, a user can access all computers and services for which he is authorized without having to log in each time.
- ▶ After successful authentication, the user receives a digital token that is used as a digital ID for applications.
- ▶ Advantages
 - ▶ user only must log in once
 - ▶ password only must be transmitted once, user only needs one password
 - ▶ withdrawal/blocking of a user is possible at a central location
- ▶ Disadvantages
 - ▶ SSO server is weak point of the system
 - ▶ availability: in case of failure all services are blocked
 - ▶ if SSO identity is stolen, access to many systems is possible





Other aspects of Single Sign On

- ▶ **Federated SSO:** trust between Identity Provider and Service Provider
- ▶ **Identity brokering:** trust between Identity Providers
- ▶ **Single Sign Out:** users are logged out of all other SSO services by logging out once.
- ▶ **Examples for SSO**
 - ▶ Kerberos
 - ▶ OpenID
 - ▶ Google
 - ▶ Windows Azure Active Directory
 - ▶ Shibboleth (open source SSO based on SAML)
 - ▶ SAML (Security Assertion Markup Language)



(Figure generated by ChatGPT)



Kerberos Authentication System



- ▶ Authentication protocol used with Windows
- ▶ Kerberos enables Single Sign On (SSO)
- ▶ No transmission of user passwords, no use of asymmetric procedures, uses symmetric procedures
- ▶ The authentication service is provided by a trusted server **Authentication Server (AS)**
- ▶ A **Ticket Granting Server (TGS)** grants tickets for access to services
- ▶ Problem: AS and TGS must always be **available online**
- ▶ Two tasks:
 - ▶ authenticate subjects (AS)
 - ▶ exchange session keys (TGS)



Kerberos: How it works

Three-steps:

- 1) Message exchange with AS (Kerberos Server)
with key for AS
→ **key for TGS**
once at login
- 2) Communication with Ticket Granting Server (TGS)
→ **tickets on demand**
- 3) Communicate with the service
→ **key for Service**

pre generated one time



Client (C)

e.g. for one day



Session key
Signs exchanges
between C and TGS

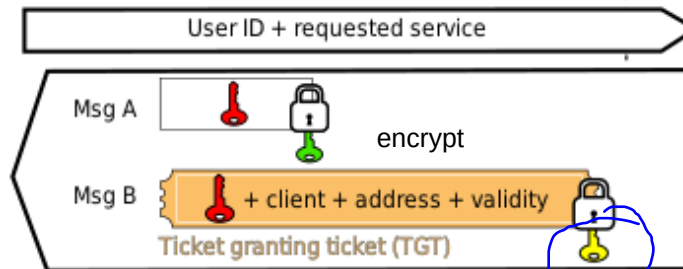
encrypted by red key



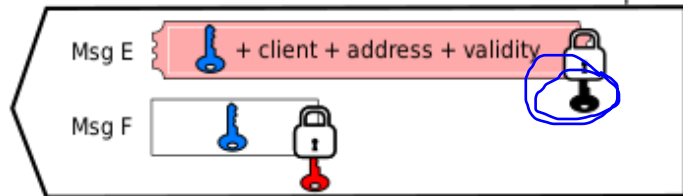
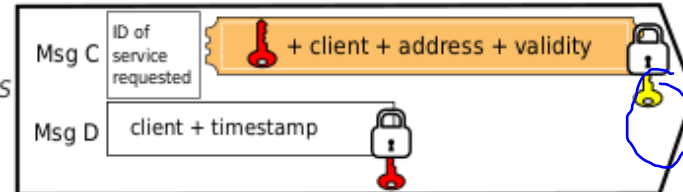
For exchanges
between C and S

session e.g. 1 hour

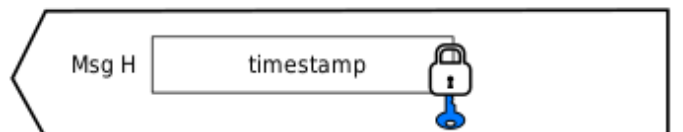
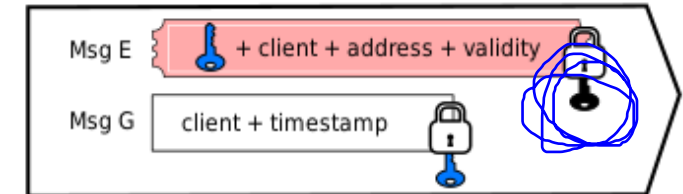
Client Authentication to the AS



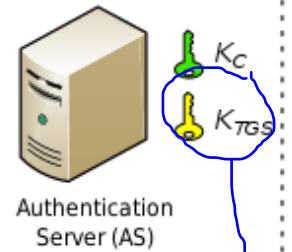
Client Service Authorization



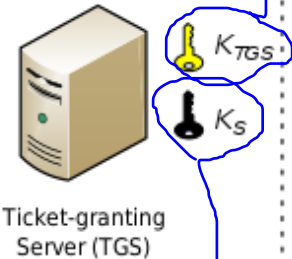
Client Service Request



Key Distribution Center (KDC)



Authentication Server (AS)



Ticket-granting Server (TGS)

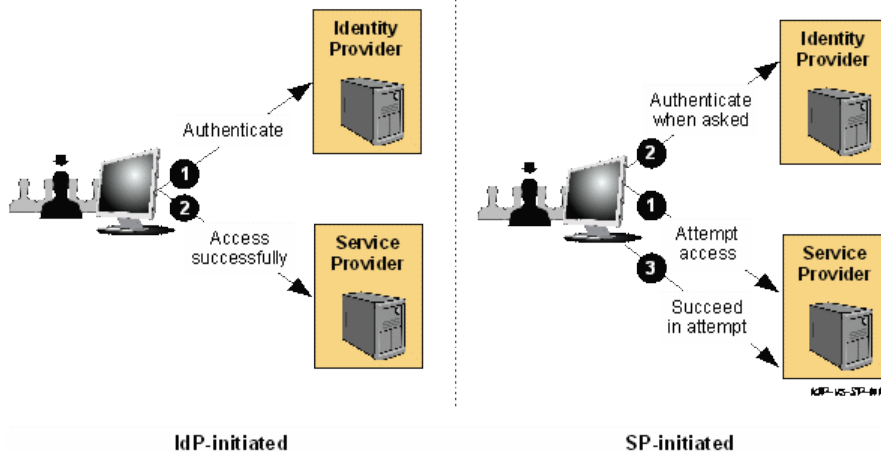


Service Server (SS)

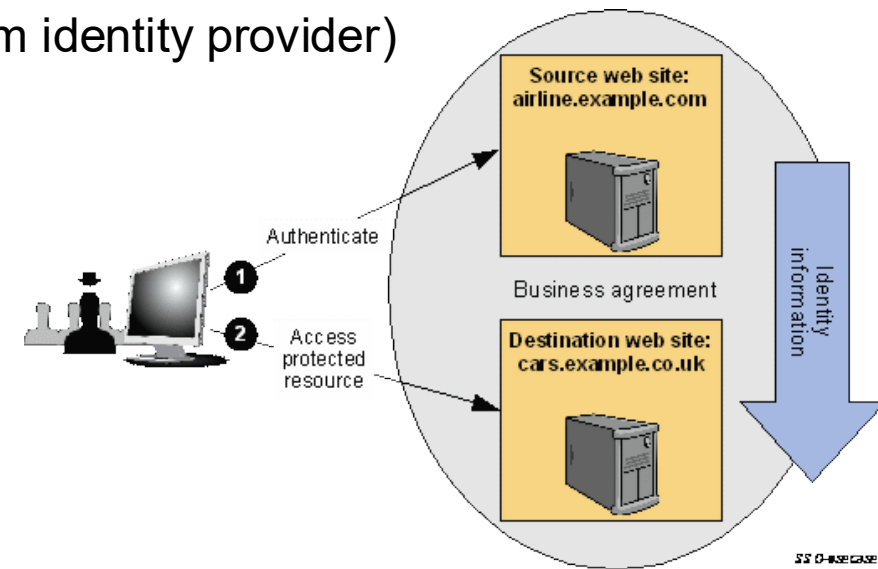
Source: https://en.wikipedia.org/wiki/Kerberos_%28protocol%29

▶ Security Assertion Markup Language (SAML)

- ▶ **SAML 2.0** is an XML framework for the exchange of authentication and authorization information (OASIS standard)
- ▶ Drivers of SAML: SSO, Federated Identity, Web services (SaaS)
- ▶ Two main types of SAML providers
 - ▶ identity provider (verifies end user)
 - ▶ service provider (requires authentication from identity provider)



Differences in Initiation of Web Browser SSO



General Single Sign-On Use Case

Quelle: <https://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0.html>



SAML Assertions

Quelle: <https://docs.oasis-open.org/security/saml/Post2.0/ssstc-saml-tech-overview-2.0.html>

```

1: <saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
2:   Version="2.0"
3:   IssueInstant="2005-01-31T12:00:00Z">
4:   <saml:Issuer Format="urn:oasis:names:SAML:2.0:nameid-format:entity">
5:     http://idp.example.org
6:   </saml:Issuer>
7:   <saml:Subject>
8:     <saml:NameID
9:       Format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress">
10:      j.doe@example.com
11:     </saml:NameID>
12:   </saml:Subject>
13:   <saml:Conditions>
14:     NotBefore="2005-01-31T12:00:00Z"
15:     NotOnOrAfter="2005-01-31T12:10:00Z">
16:   </saml:Conditions>
17:   <saml:AuthnStatement>
18:     AuthnInstant="2005-01-31T12:00:00Z" SessionIndex="67775277772">
19:     <saml:AuthnContext>
20:       <saml:AuthnContextClassRef>
21:         urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport
22:       </saml:AuthnContextClassRef>
23:     </saml:AuthnContext>
24:   </saml:AuthnStatement>
25: </saml:Assertion>
  
```

Figure 6: Assertion with Subject, Conditions, and Authentication Statement

SAML assertions are statements about **properties** (identity, attributes) and **permissions** of a user

name
friendly name

Credit limit

```

1: <saml:AttributeStatement>
2:   <saml:Attribute
3:     xmlns:x500="urn:oasis:names:tc:SAML:2.0:profiles:attribute:X500"
4:     NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:uri"
5:     Name="urn:oid:2.5.4.42"
6:     FriendlyName="givenName">
7:     <saml:AttributeValue xsi:type="xs:string"
8:       x500:Encoding="LDAP">John</saml:AttributeValue>
9:   </saml:Attribute>
10:   <saml:Attribute
11:     NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:basic"
12:     Name="LastName">
13:     <saml:AttributeValue
14:       xsi:type="xs:string">Doe</saml:AttributeValue>
15:   </saml:Attribute>
16:   <saml:Attribute
17:     NameFormat="http://smithco.com/attr-formats"
18:     Name="CreditLimit">
19:     xmlns:smithco="http://www.smithco.com/smithco-schema.xsd"
20:     <saml:AttributeValue xsi:type="smithco:type">
21:       <smithco:amount currency="USD">500.00</smithco:amount>
22:     </saml:AttributeValue>
23:   </saml:Attribute>
24: </saml:AttributeStatement>
  
```

Figure 7: Attribute Statement



OAuth (Open Authorization)

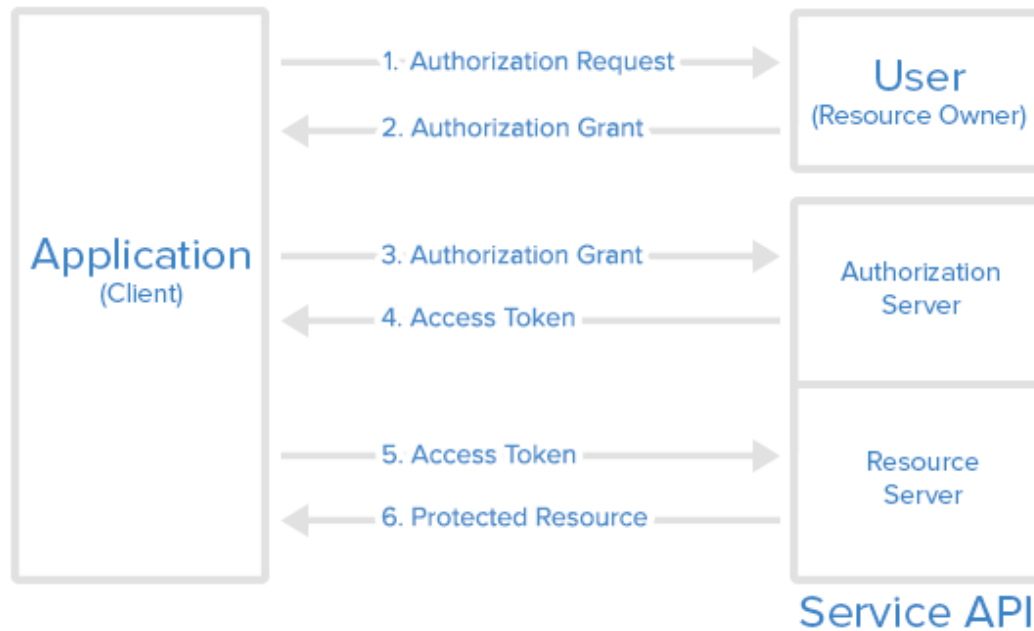


- ▶ **OAuth 2.0** is a standard for the authorizing of API accesses on the web
- ▶ OAuth 2.0 enables delegation of authorization.
- ▶ An **access token** contains authorization for a client (e.g., customer) and a set of permissions.
- ▶ A **refresh token** can be used to obtain a new access token
- ▶ OAuth 2.0 is very well suited for server-side applications (e.g. cloud native services) where the access tokens can be stored securely
 - ▶ owner grants one-time permission to invoke a service
 - ▶ server stores access and refresh tokens



OAuth 2.0 authorization protocol flow

Abstract Protocol Flow



Answer of authorization server with **access token** and **refresh token**

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: no-cache
{
  "access_token":"2YotnFZFEjr1zCsicMWpAA",
  "token_type":"example",
  "expires_in":3600,
  "refresh_token":"tGzv3JOkF0XG5Qx2TIKWIA",
  "example_parameter":"example_value"
}
```

Source: <https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

<https://tools.ietf.org/html/rfc6749#section-1.5>

Authorization Grant = permission from owner to access resource

Access Token = token to access resource



SSO in the Web



OpenID

- ▶ **OpenID Connect** extends OAuth with all necessary functions for login and SSO
- ▶ OpenID supports decentralization: there are many OpenID providers, switching between them is possible
- ▶ Authentication information is stored in the ID Token, a signed **JSON Web Token JWT**
 - ▶ Claims (UserID, email, ...)
 - ▶ Metadata
 - ▶ Optional Encryption



Simple & Mobile Friendly

JSON Based

REST Friendly

In simplest cases,
just copy and paste

Mobile & App
Friendly

e.g., ID Token is signed JSON

```
{
  "iss": "https://client.example.com",
  "sub": "24400320",
  "aud": "s6BhdRkqt3",
  "nonce": "n-0S6_WzA2Mj",
  "exp": 1311281970,
  "iat": 1311280970,
  "auth_time": 1311280969,
  "acr": "2",
  "at_hash":
    "MTIzNDU2Nzg5MDEyMzQ1Ng"
}
```



Quelle:

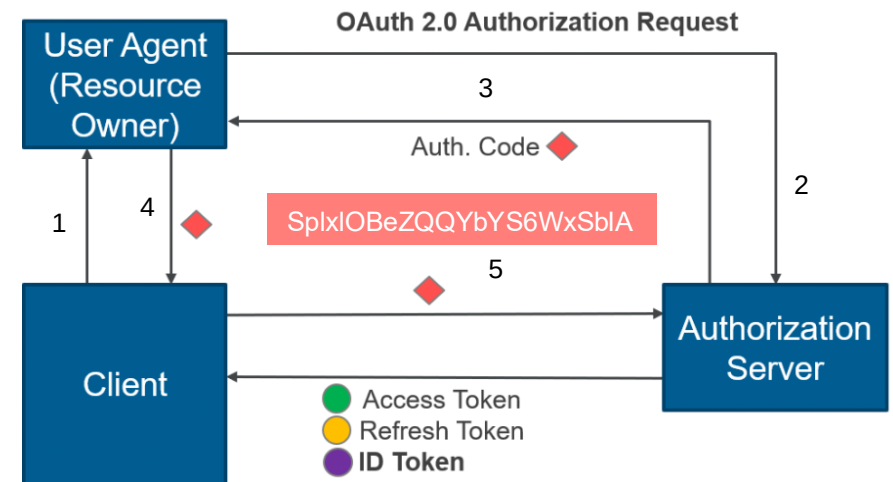
http://de.slideshare.net/nat_sakimura/introduction-to-openid-connect/30



The interaction of OAuth, OpenID Connect and JWT

- ▶ Resource Owner gives Client the **Authorization Code** via an OAuth Request
- ▶ Client fetches the tokens to access the resource with Authorization Code
- ▶ **Access Token**: allows access to the resource and the userinfo
- ▶ **Refresh Token**: is used to renew an expired Access Token
- ▶ **ID Token** contains identity information about a user and other user attributes (claims) in the format JWT

```
https://account-d.docusign.com/oauth/auth?
response_type=code
&scope=YOUR_REQUESTED_SCOPES
&client_id=YOUR_INTEGRATION_KEY
&state=YOUR_CUSTOM_STATE
&redirect_uri=YOUR_REDIRECT_URI
&login_hint=YOUR_LOGIN_EMAIL
```



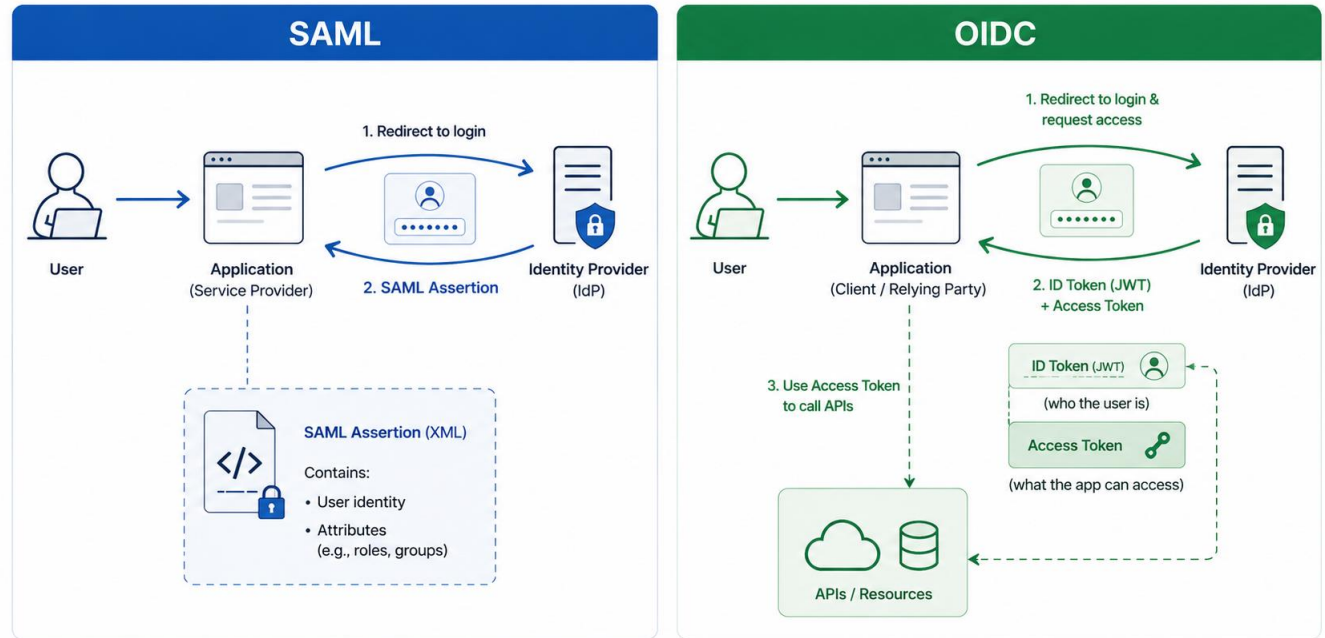
```
{
  "access_token": "ISSUED_ACCESS_TOKEN",
  "token_type": "Bearer",
  "refresh_token": "ISSUED_REFRESH_TOKEN",
  "expires_in": 28800
}
```

Quelle <https://www.oose.de/blogpost/oauth-openid-connect-und-jwt-wie-haengt-das-alles-zusammen-teil-2/>



SAML vs OIDC

SAML vs OIDC



(Figure generated by ChatGPT)

	SAML	OIDC
Focus	Authentication (SSO) authorizations SAML	Authentication (OIDC) & Authorization (OAuth2)
Format	XML	JSON (JWT)
Protocol	HTTP Redirect / POST	HTTP / REST
Token	SAML Assertion	ID Token (JWT) , Access Token
Typical use	Enterprise SSO	Modern Apps, APIs



Chapter 4.3: Authorization

- DAC, MAC
- RBAC
- RuBAC
- ABAC
- Access Control Patterns

Authorization – models of access control

combinations are possible

▶ Discretionary-Access-Control (DAC)

social networks (who can see your data)

- ▶ user-definable access control Unix -> user can delegate rights to another user
- ▶ each owner can transfer rights to his objects to other users
- ▶ the assignment of rights is controlled decentral
- ▶ restricting access to objects based on the identity of the subject

▶ Mandatory Access Control (MAC)

- ▶ system defines security properties (rule-based)
- ▶ user-defined rights are overridden (dominated) by system-defined ones
- ▶ additional security classes and global rules are introduced
- ▶ limiting access to resources is based on the sensitivity of the information
- ▶ operating systems or applications must provide special measures and services to enforce MAC policies

e.g. military systems, strictly confidential systems



Realization of access control via access matrix

combinations are possible

- ▶ Access control is often implemented with a (sparse) access matrix that can be viewed in two dimensions

		Objekte				
		o1	o2			on
Subjekte	s1	r1				
	s2	r2	r1			
	sn					r2

Zugriffsrechte

- ▶ Access control list - **Access Control Lists (ACL)**
 - ▶ object-based view, one list per object to be protected
 - ▶ ACL define the access rights of subjects to objects
 - ▶ advantage: easy administration and revocation of rights
 - ▶ disadvantage: sometimes inefficient with many subjects

ACL (file1) = ((user 1, r), (user 2, x))

- ▶ Access tickets - **Capabilities (permissions)**

CAP(user 1) = ((file1, r), (file2, d))

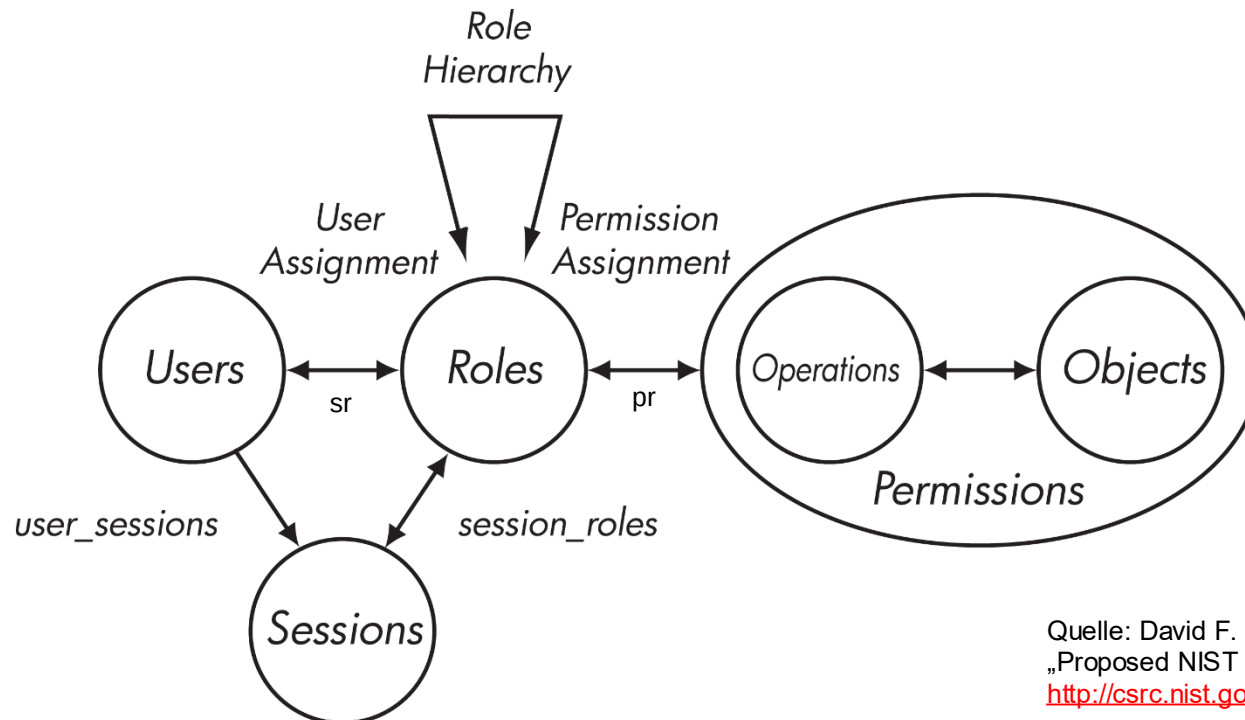
- ▶ subject-based view
- ▶ Tamper-proof tickets that authorize the holder to access an object
- ▶ advantage: flexible, decentralized, suitable for delegation
- ▶ disadvantage: withdrawal of rights is time-consuming



Role Based Access Control (RBAC)

▶ Rolle-Based-Access-Control-Pattern

- ▶ the permissions for objects are assigned to roles
(**pr = permission to role**)
- ▶ subjects are assigned to roles
(**sr = subject to role**)



Quelle: David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, R. Richard Kuhn, „Proposed NIST Standard for Role-Based Access Control“, 2001
<http://csrc.nist.gov/rbac/rbacSTD-ACM.pdf>



Components of a RBAC model

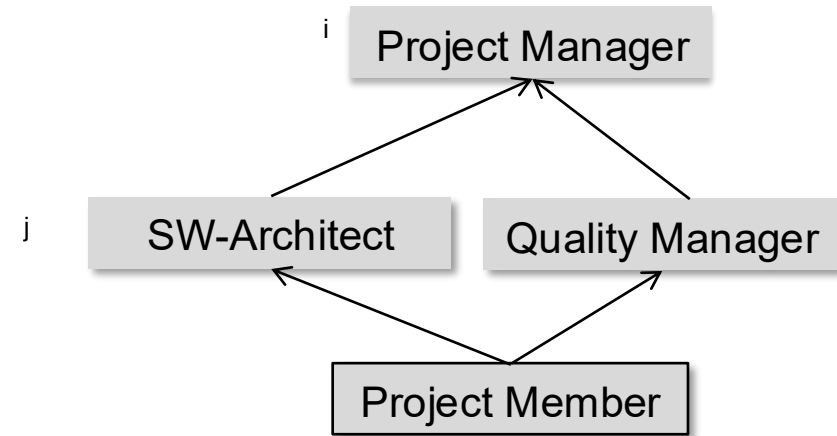
▶ Sessions

- ▶ a session means a subject is active in a role
- ▶ a subject may only be active in roles of which he is a member
- ▶ a subject has only the rights of his active role

▶ Role hierarchy

- ▶ goal : replication of organizational structures
- ▶ definition of a partial order on roles

$R_i, R_j \in Role, \text{if } R_i \leq R_j$
then R_i has all rights of R_j



- ▶ **Static separation of duties**: mutual exclusion of role memberships.

What are the benefits of RBAC-Model?

- ▶ Role concepts are very **flexible** to use, task-oriented, **administrable** and **scale** well
- ▶ They allow direct replication of known **organizational and rights structures** in companies and are a good basis for ID management
- ▶ Intuitive and relatively simple mapping of roles to **business processes** (workflows) enables **need-to-know** rights assignment
- ▶ Changes to **pr** are rare;
but changes to role memberships **sr** are frequent;
- ▶ Simple and efficient rights management, automatic rights revocation at end of membership.
- ▶ Danger: Roles are misused for **highly specific permissions**, which can lead to an exploding number of roles



Rule-Based Access Control (RuBAC)

- ▶ Access control based on rules (**policies**)

firewalls
routers

- ▶ Typical applications:
Firewalls, routers

- ▶ With user rights it can be used with MAC
 - ▶ rules describe situations in which a subject can access an object
 - ▶ RuBAC quickly become very complex

- ▶ Rules can be described with policies:
Policy-Based-Access Control PBAC

```
# Allow users to get their own salaries.
allow {
  input.method = "GET"
  input.path = ["finance", "salary", username]
  input.user == username
}

# Allow managers to get their subordinates' salaries.
allow {
  input.method = "GET"
  input.path = ["finance", "salary", username]
  subordinates[input.user][_ ] == username
}
```

Example of an Open Policy Agent in Policy language Rego („ray-go“)

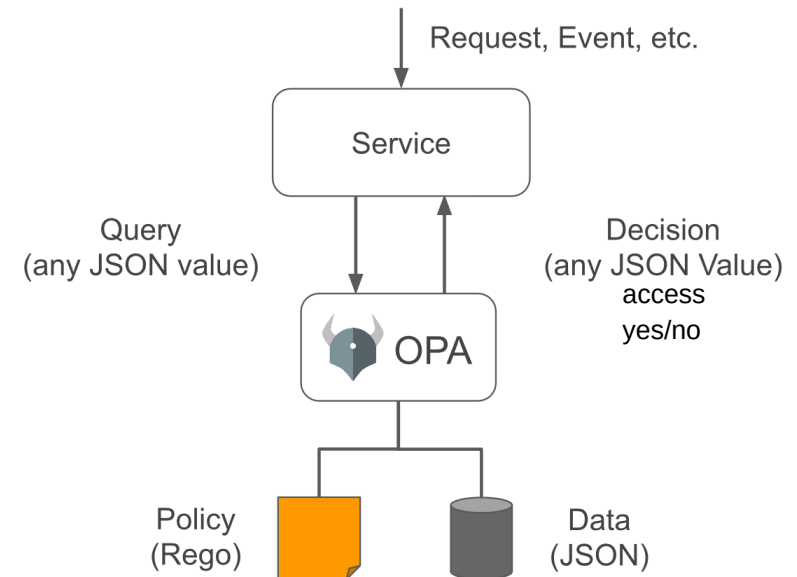


Open Policy Agent



Open Policy Agent OPA enables Rule Based Access Control in cloud environments

check your rules

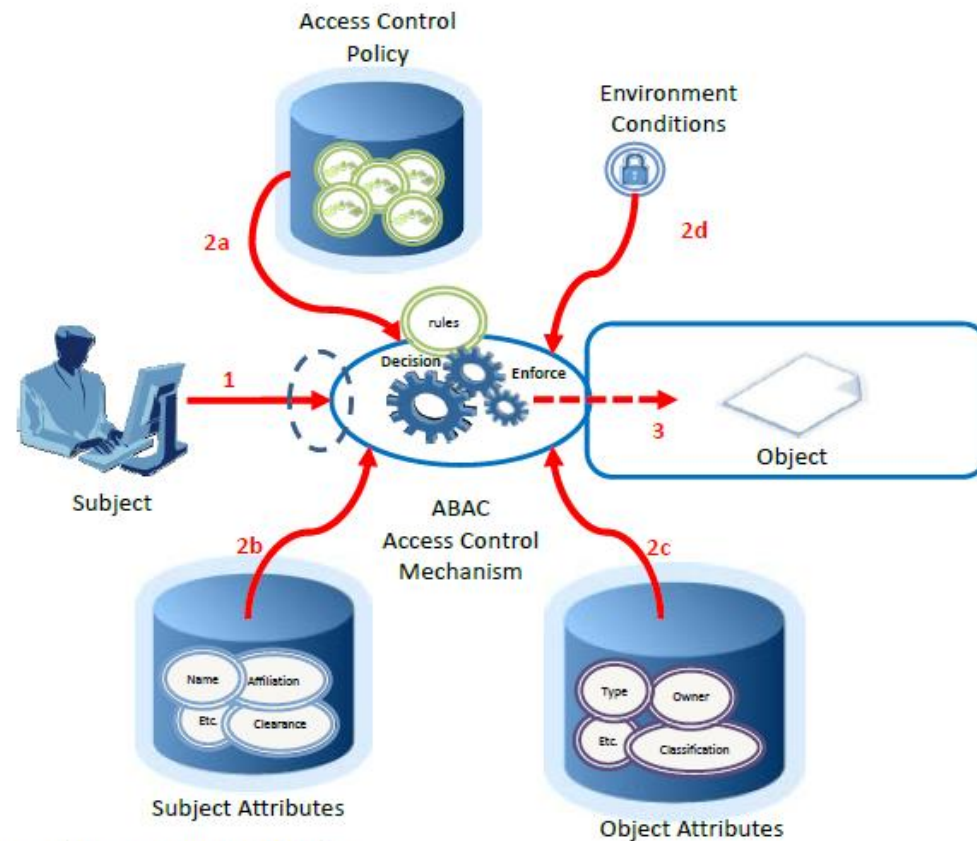


OPA generates policy decisions by evaluating the query input and against policies and data

OPA enables fine-grained policy-based control in cloud native environments

Quelle: <https://www.openpolicyagent.org/>

▶ Attribute Based Access Control (ABAC)



1. Subject requests access to object
2. Access Control Mechanism evaluates a) Rules, b) Subject Attributes, c) Object Attributes, and d) Environment Conditions to compute a decision
3. Subject is given access to object if authorized

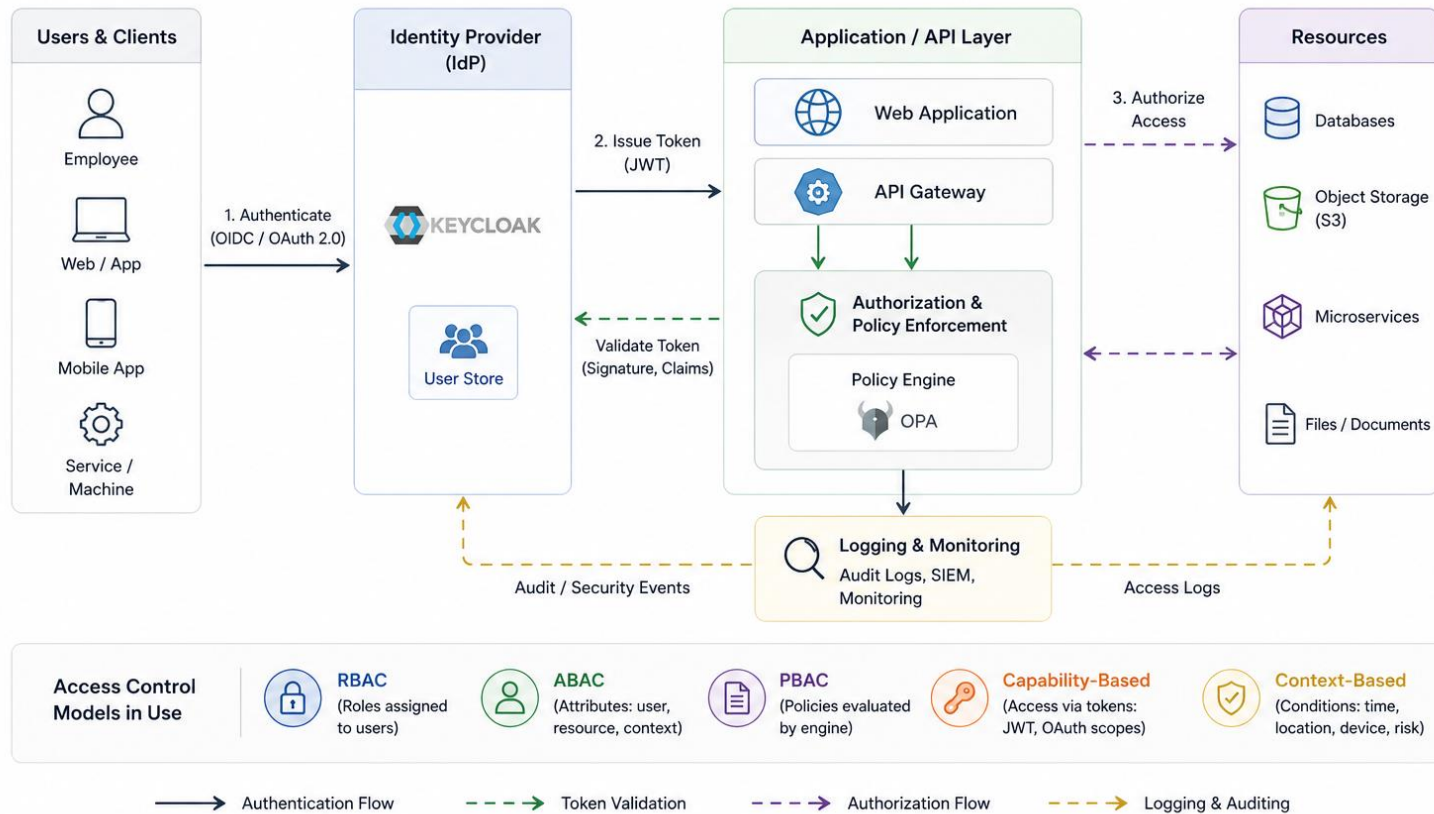
Quelle: NIST <http://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.sp.800-162.pdf>

- ▶ Decisions based on attribute (user, object, environment)
- ▶ OpenID Connect enables access control based on attributes (claims)
- ▶ XACML eXtensible Access Control Markup Language is an attribute-based access control policy language.
- ▶ Application area:
 - ▶ API gateway for micro services
 - ▶ Access to Big Data systems

Access Control Patterns can be combined

Access Control in a Modern Cloud Architecture

How different access control models work together



(Figure generated by ChatGPT)

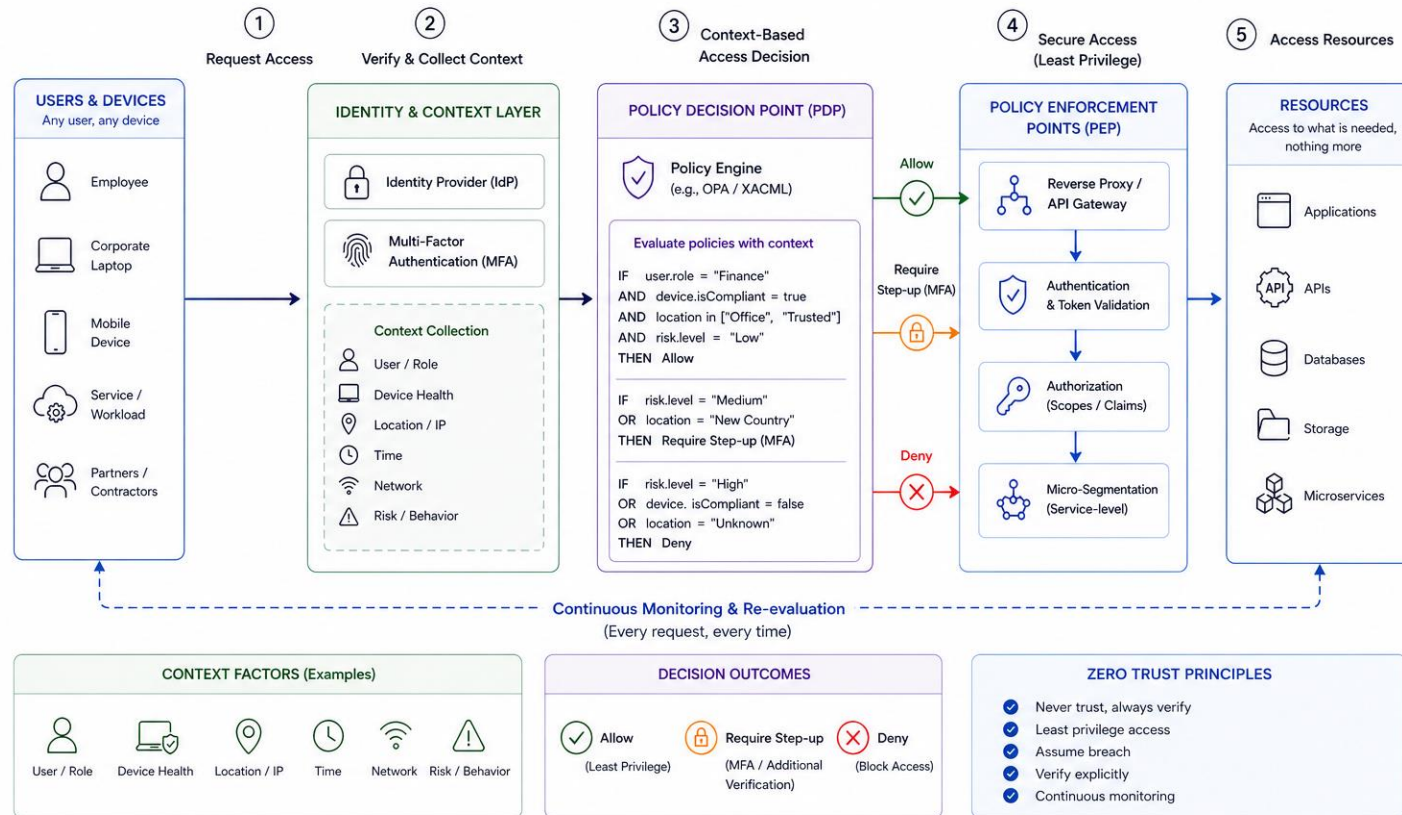


Zero Trust Architecture: “Never trust, always verify”

Replaces the traditional security: Everything inside the network is trusted

Zero Trust Architecture

Context-Based Access Control at Every Step



(Figure generated by ChatGPT)



Access Control Patterns

- ▶ **Least privilege:** a subject should be given only those privileges needed for it to complete its tasks, raises system stability and security
- ▶ **Need to Know:** user gets access only if it's necessary to conduct its duties
- ▶ **Separation of Duty:** more than one user is required to complete a task, increases protection from fraud and errors, control against insider attacks
- ▶ **Separation of Concerns:** separate a computer program into distinct sections
- ▶ **Open Policy:** everything is allowed which is not forbidden
- ▶ **Closed Policy:** only explicit authorized access is allowed
- ▶ **Dual Control:** Four eyes principle, two or more separate entities are necessary to access sensitive functions or information

Summary Authentication and Authorization



- ▶ There are many different variants for authentication
- ▶ They differ in security, mobility, cost and convenience
- ▶ There are different standards and technologies for SSO (Kerberos, OAuth, OpenID, SAML)
- ▶ There are many methods of access control
- ▶ RBAC is a very flexible model for managing access rights
- ▶ In modern cloud environments, RuBAC and ABAC also play an important role